Deep Learning for Economics and Finance

Histogram non-stochastic distribution updates, Krusell–Smith, continuum-of-agents DEQN

Simon Scheidegger

HEC, University of Lausanne

Based on: Young (2010), *J. Econ. Dynamics & Control* 34, 36–41;
Azinovic, Gaegauf & Scheidegger (2022), *International Economic Review* 63(4), 1471–1525.

Materials: `https://github.com/sischei/Deep_Learning_for_Solving_And_Estimating_Dynamic_Economic_Models`

Additional materials / further reading

Self-study extension

- The discrete-time lectures covers **Automatic Differentiation**, **Sequence-Space DEQNs**, and **OLG models** with DEQNs.
- This Young's-method deck, together with the companion notebooks `10_Youngs_Method_Examples`, `11_Continuum_of_Agents_DEQN`, and `12_KrusellSmith_DeepLearning`, is provided as **additional materials / further reading** for heterogeneous-agent solution methods.
- The slides are fully self-contained: motivation, the Young (2010) histogram update operator, the Krusell–Smith benchmark, and the Appendix A.5 DEQN teaching implementation.
- Students interested in heterogeneous-agent work in discrete time should read this deck alongside the PINN-based continuous-time treatment.

Outline

1. **Motivation: Why Track Distributions?**
   From representative agents to heterogeneous agents; the infinite-dimensional state space.
2. **Young's (2010) Non-Stochastic Simulation**
   Histogram representation, mass redistribution, mean preservation, full algorithm.
3. **Benchmark: Krusell–Smith (1998)**
   Approximate aggregation, the forecasting-rule benchmark, and where Young fits.
4. **DEQN Teaching Implementation**
   Appendix A.5 endowment economy, histogram encoding, network architecture, training loop.
5. **Alternative Deep-Learning Approaches to KS**
   All-in-one DL (Maliar–Maliar–Winant, 2021) and DeepHAM (Han–Yang–E, 2024).

Learning objectives

By the end of this lecture you should be able to:

- Recognize why aggregate state becomes infinite-dimensional in heterogeneous-agent economies and how Young's method avoids the curse
- Implement a histogram-based distribution simulator that updates aggregate states without Monte Carlo noise
- Build and train a policy network on a two-dimensional wealth distribution in the Krusell–Smith benchmark
- Diagnose the failure mode of state-space deep learning when the distribution shifts and explain how sequence space addresses it
- Deploy shape-preserving (I-spline) parameterizations to enforce marginal-propensity-to-consume monotonicity by construction

Part I

Motivation: Why Track Distributions?

From Representative Agent to Heterogeneous Agents

the discrete-time lectures recap:

- Brock–Mirman: single agent, single capital stock
- IRBC: N countries, but a social planner with complete markets
- State: $(k^1, \ldots, k^N, z^1, \ldots, z^N)$—finite-dimensional

$\Rightarrow$ All agents are "representative."

Reality:

- Agents differ in wealth, income, employment status
- Incomplete markets: cannot insure against idiosyncratic risk
- Aggregate shocks affect agents differently

Key questions:

- How does aggregate uncertainty affect inequality?
- How does the wealth distribution feed back into prices?

Why Distributions Matter

Policy implications:

- Fiscal stimulus: who spends the transfer?
- Monetary policy: affects borrowers and savers differently
- Wealth distribution shapes aggregate demand
- Consumption response depends on position in distribution

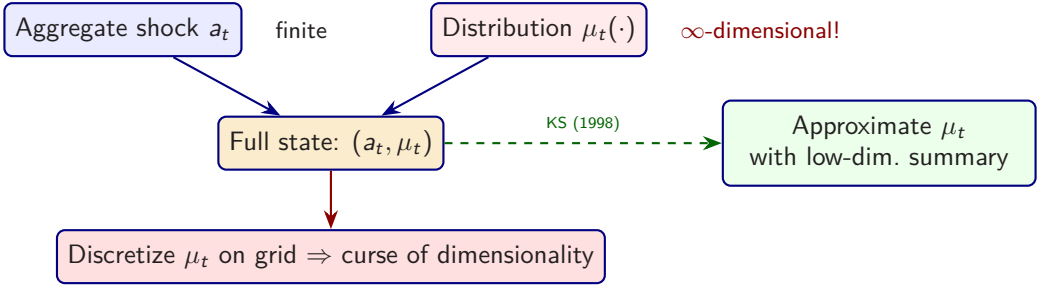
Agents near the borrowing constraint have high MPC; wealthy agents save most of a windfall.

Mathematical challenge:

- The entire distribution μ_t is a state variable
- μ_t is a measure over individual states
- Infinite-dimensional object!
- Cannot put it on a grid without the curse of dimensionality

$\Rightarrow$ Need clever methods to approximate μ_t.

The Infinite-Dimensional State Space Problem



- Traditional methods: discretize the distribution $\Rightarrow$ exponential cost
- Key insight (Krusell–Smith): low-order moments may suffice

Two Approaches to Heterogeneous Agents

Approach 1: Finitely Many Agents

Track N agents explicitly: $(k_1, \ldots, k_N, \varepsilon_1, \ldots, \varepsilon_N)$

Pro:

- Exact—no approximation of μ_t

Con:

- Permutation problem: agents are exchangeable
- State space grows with N
- Scales poorly for large N

Maliar, Maliar & Valli (2010)

Approach 2: Continuum of Agents

Track the distribution μ_t directly

Pro:

- Law of large numbers $\Rightarrow$ no sampling noise
- No permutation issue
- Distribution is order-invariant

Con:

- Must approximate μ_t (infinite-dimensional)

Krusell & Smith (1998), Young (2010)

$\Rightarrow$ **This lecture:** Approach 2 with Young's histogram method + DEQNs.

The Krusell–Smith Insight (1998)

Key insight: the mean of the wealth distribution is *nearly sufficient* to predict future prices.

Approximate bounded rationality: agents forecast using $m_1 = \int k \, d\mu$ (mean capital).

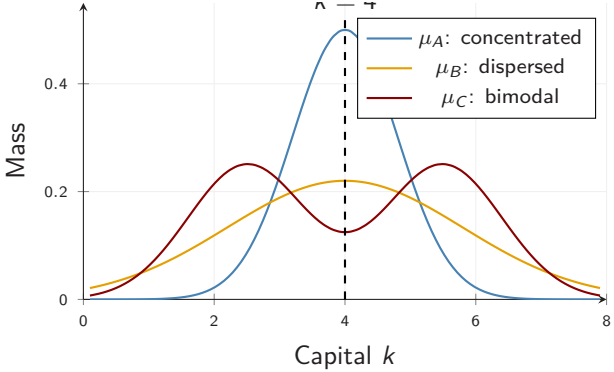
Log-linear forecasting rule:

$$\boxed{\hat{H}(m_1, a) = \exp\big(A(a) + B(a) \log m_1\big)}$$

- Coefficients $A(a), B(a)$ depend on the aggregate shock a
- Inada conditions ensure capital remains positive and logs are well-defined
- $R^2 > 0.999$ in practice—remarkably accurate!

But how do we simulate the distribution forward to compute realized m_1?

Why the Mean Is (Nearly) Sufficient



Bond prices:

$p(\mu_A) = 0.981$
$p(\mu_B) = 0.979$
$p(\mu_C) = 0.980$

Nearly identical!

Same mean $\bar{k} = 4$

$\Rightarrow$ similar prices

Three very different distributions that share the same mean capital $\bar{k} = 4$. In the KS economy, the resulting bond prices differ by $< 0.3\%$. This is why the mean is a nearly sufficient statistic for forecasting—and why approximate aggregation works so well.

But How Do We Simulate the Distribution?

Given policy functions, need to compute the next-period distribution μ_{t+1}.

Option A: Monte Carlo simulation

- Draw N agents, simulate forward, compute sample moments
- Problem: sampling noise $\sim O(1/\sqrt{N})$
- Need very large N ($> 50{,}000$) for accurate means
- Noise accumulates over time

Option B: Non-stochastic simulation (Young, 2010)

- Track the histogram directly on a fixed grid
- No sampling error—deterministic update
- Preserves the mean exactly
- Computationally fast

$\Rightarrow$ This lecture focuses on **Option B**: Young's (2010) method

Where Are We?

1. Motivation: Why Track Distributions? ✓
2. ⇒ Young's (2010) Non-Stochastic Simulation
3. Benchmark: Krusell–Smith (1998)
4. DEQN teaching implementation
5. Alternative deep-learning approaches to KS

Part II

Young's (2010) Non-Stochastic Simulation

The Core Idea

Instead of simulating a panel of agents $\longrightarrow$ simulate the distribution directly.

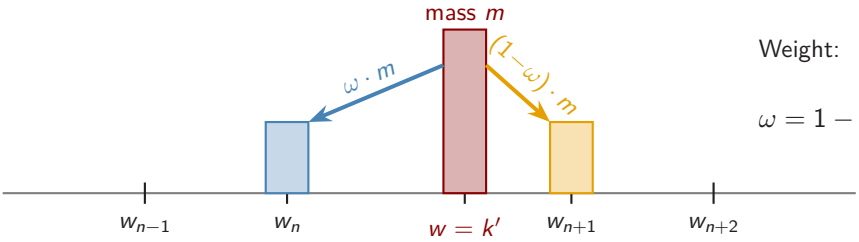
Setup:

1. Choose a fixed grid: $[w_1, w_2, \ldots, w_N]$
2. Represent the distribution as a histogram: mass $G(w_i)$ at each grid point
3. Update the histogram deterministically using:
   - Policy functions $k' = g(k, \varepsilon, m_1, a)$
   - Transition probabilities $\pi_{\varepsilon'|\varepsilon,a}$

Key insight

When k' falls between grid points, redistribute the mass to the two nearest grid points using linear interpolation weights. This preserves the mean exactly.

Mass Redistribution: The Key Operation



Weight:

$$\omega = 1 - \frac{w - w_n}{w_{n+1} - w_n}$$

- Mass m at $w \in [w_n, w_{n+1}]$ is split between the two bracketing grid points
- Weight ω: fraction to w_n (closer $\Rightarrow$ more mass)
- Weight $1 - \omega$: fraction to w_{n+1}

Lottery View: Why "Non-Stochastic"

The redistribution *is* a lottery

Every agent at (k, ε) whose policy choice k' lands between k_n and k_{n+1} is reassigned by a biased coin:

$$k_{t+1} = \begin{cases} k_n & \text{with probability } \omega, \\ k_{n+1} & \text{with probability } 1-\omega, \end{cases} \qquad \omega = 1 - \frac{k' - k_n}{k_{n+1} - k_n}.$$

Monte Carlo panel

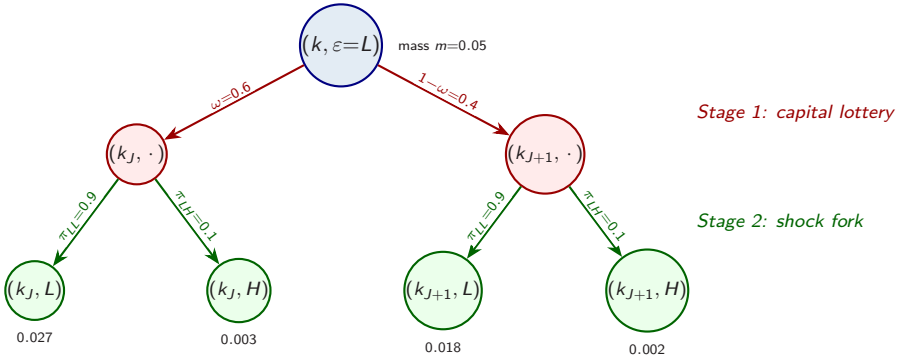
Each of N agents draws the lottery once.
Empirical histogram converges at rate $\mathcal{O}(N^{-1/2})$.
Sampling noise contaminates aggregates.

Young's method

Integrates the lottery analytically.
Equivalent to running $N = \infty$ agents.
Zero noise; mean preserved exactly.

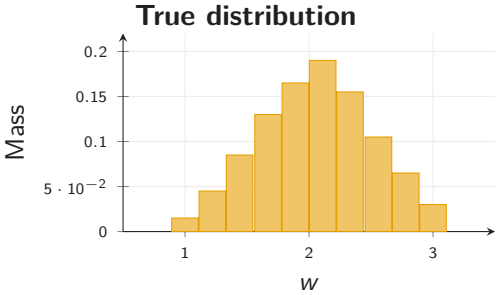
The same trick is applied to the shock transitions $\pi_{\varepsilon\varepsilon'}$: each piece of mass goes into *all* reachable next-period ε' in proportion to π, instead of one realisation being drawn.

One Cell Splits Into Four (Young, Fig. 1)

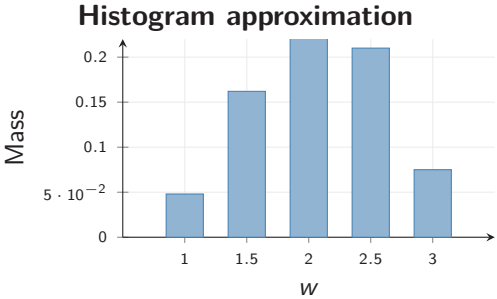


Verification: $0.027 + 0.003 + 0.018 + 0.002 = 0.05 = m$ ✓. Repeat for every active source bin and accumulate the leaves to obtain G_{t+1}.

Example: Multi-Point Distribution



10 points with truncated normal masses



5-point grid approximation

	True	Approximation
Mean	2.0	2.0 (exact!)
Variance	0.347	0.384 (approx.)

Key Property: Mean Preservation

The weight ω is forced by mean preservation

ω is the unique value for which the two-point split $\{\omega \cdot w_n, (1-\omega) \cdot w_{n+1}\}$ has expected value equal to the off-grid policy w. Solving $\omega \cdot w_n + (1-\omega) \cdot w_{n+1} = w$ for ω gives back $\omega = 1 - (w - w_n)/(w_{n+1} - w_n)$ in one line:

$$\omega \cdot w_n + (1-\omega) \cdot w_{n+1} \;=\; w_n + (1-\omega)(w_{n+1} - w_n) \;=\; w_n + (w - w_n) \;=\; w. \quad \checkmark$$

- Total mass preserved automatically: $\omega + (1-\omega) = 1$.
- By linearity of expectation, the unconditional mean of G_{t+1} equals $\mathbb{E}[k']$ *exactly*, provided no mass leaks past the grid endpoints.

Caution

Higher moments (variance, percentiles) are only *approximated*—finer grid = better approximation. Linear interpolation is exact for linear functions but smooths nonlinear ones.

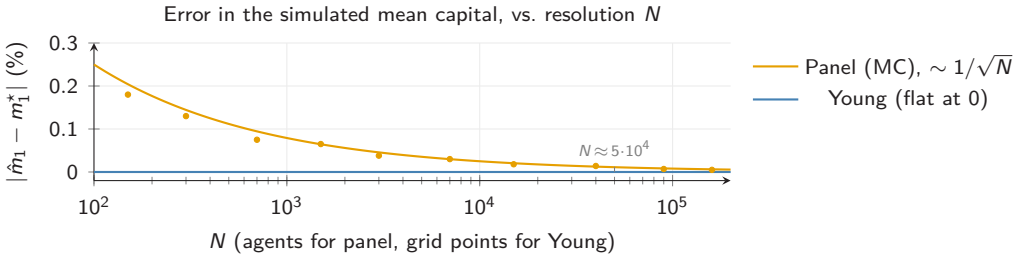
Why Not Just Use a Panel?

	Young's Method	**Panel Simulation**
Sampling noise	None	$O(1/\sqrt{N})$
Mean preservation	Exact	Approximate
Agents/points needed	$\sim$100–5000 grid pts	$>$50,000 agents
Speed	Fast (deterministic)	Slower (stochastic)
Memory	Fixed grid	Growing with N
Reproducibility	Deterministic	Seed-dependent

- Mean preservation is the headline property: each Young update reassigns mass deterministically using the lottery weight ω, so the new mean equals the policy-implied mean exactly (boundary clipping aside).
- In a Monte-Carlo panel the same lottery is *drawn once per agent*, so the realized mean is noisy at rate $1/\sqrt{N}$.

$\Rightarrow$ The next slide quantifies the gap.

Convergence: Young vs. Monte-Carlo Panel



What is plotted. Deviation of the simulated mean capital $\hat{m}_1$ from its true value $m_1^\star$, as the resolution N grows. Orange dots: realized Monte-Carlo errors of a panel of N agents, scattered around the $1/\sqrt{N}$ envelope. Blue line: Young's deterministic update, which preserves the mean exactly at *every* grid resolution.

Implication. To match Young's accuracy at $\sim$1,000 grid points, a Monte-Carlo panel needs $\gtrsim$ 50,000 agents (an order of magnitude more state), and the residual noise still contaminates the OLS step that updates the KS forecasting rule.

The Algorithm: Setup

Ingredients:

- **Asset grid:** $[k_1, k_2, \ldots, k_{N_k}]$ for individual capital/asset holdings
- **Idiosyncratic states:** $\varepsilon \in \{e, u\}$ (employed/unemployed)
- **Histogram:** $G_t(k_i, \varepsilon_j)$ = mass of agents at grid point k_i with shock ε_j
- **Total bins:** $N_k \times N_\varepsilon$ histogram entries

Normalization:

$$\sum_{i=1}^{N_k} \sum_{j=1}^{N_\varepsilon} G_t(k_i, \varepsilon_j) = 1$$

First moment (mean capital):

$$m_1 = \sum_{i=1}^{N_k} \sum_{j=1}^{N_\varepsilon} G_t(k_i, \varepsilon_j) \cdot k_i$$

The Algorithm: One Forward Step

Algorithm: Histogram Update at Time t

Input: Current histogram G_t, policy $g(\cdot)$, transition $\pi_{\varepsilon'|\varepsilon,a}$, grid $\{k_j\}$
Initialize $G_{t+1}(k_j, \varepsilon') = 0$ for all (j, ε')
for each (k_i, ε_j) with $G_t(k_i, \varepsilon_j) > 0$ **do**
  Compute optimal savings: $k' = g(k_i, \varepsilon_j, m_1, a_t)$
  Find bracketing grid points: $k' \in [k_J, k_{J+1}]$
  Compute lottery weight: $\omega = 1 - \frac{k' - k_J}{k_{J+1} - k_J}$
  for each next-period shock ε' **do**
    $G_{t+1}(k_J, \varepsilon') \mathrel{+}= \omega \cdot \pi_{\varepsilon'|\varepsilon_j, a_t} \cdot G_t(k_i, \varepsilon_j)$
    $G_{t+1}(k_{J+1}, \varepsilon') \mathrel{+}= (1-\omega) \cdot \pi_{\varepsilon'|\varepsilon_j, a_t} \cdot G_t(k_i, \varepsilon_j)$
  end for
end for
Output: Updated histogram G_{t+1}

Notation note. The interpolation weight is denoted ω throughout this deck and the script. It is the probability mass sent to the lower bracketing grid point.

Implementation: Young's Update in NumPy

Four scatter-add lines (one per leaf of the cascading fork):

```
def young_step(G, kp, pi_eps, k_grid):
    """G[i,j]: mass at (k_grid[i], eps_j); kp[i,j]: policy k'."""
    G_next = np.zeros_like(G)
    dk     = k_grid[1] - k_grid[0]
    n_k    = len(k_grid)
    for j in range(G.shape[1]):
        for i in range(G.shape[0]):
            x     = np.clip(kp[i, j], k_grid[0], k_grid[-1])
            J     = min(int((x - k_grid[0]) // dk), n_k - 2)
            omega = (k_grid[J + 1] - x) / dk            # in [0, 1]
            for jp in range(G.shape[1]):
                w = pi_eps[j, jp] * G[i, j]
                G_next[J,     jp] += omega       * w
                G_next[J + 1, jp] += (1 - omega) * w
    return G_next
```

JAX vectorisation of the same operator (`distribution_step` via `jax.vmap` and `.at[].add()`) is in `KrusellSmith_Tutorial_CPU.ipynb`.

Why Closed-Form Bracketing Matters on GPUs

The lookup. Young's update — and any piecewise-linear interpolation — needs the bracketing index for each query k':

$$J(k') \;=\; \max\{\, n : k_n \le k' \,\}, \qquad k' \in [k_J,\, k_{J+1}).$$

Generic approach. Binary search (e.g. `jnp.searchsorted`): $\mathcal{O}(\log N_k)$ per query, with data-dependent branches.

Consequences on GPU / under XLA.

- Warp divergence across SIMT threads taking different branches.
- Irregular gather patterns and extra kernel launches.
- Opaque search op breaks fusion with the surrounding interpolation arithmetic.

Closed-form alternative (uniform grid).

$$J(k') \;=\; \mathrm{clip}\left(\left\lfloor \frac{k'-k_0}{\Delta k} \right\rfloor,\; 0,\; N_k - 2\right).$$

One fused multiply–add and a cast: branch-free, uniform across threads, fuses with the lottery weight $\omega = (k_{J+1} - k')/\Delta k$ into a single GPU kernel. This is exactly the line `J = int((x - k_grid[0]) // dk)` on the previous slide.

Log-Spaced Grids Without Giving Up the Closed Form

Why log-spacing. The consumption policy is steepest near the borrowing constraint $k \to 0$; we want fine resolution there without wasting points in the right tail.

Recipe. Equidistant knots in a transformed coordinate $x = \log(c + k)$ for a small shift $c > 0$:

$$x_n \;=\; x_0 + n\,\Delta x, \qquad k_n \;=\; e^{x_n} - c, \qquad n = 0, \ldots, N_k.$$

Bracketing stays closed-form.

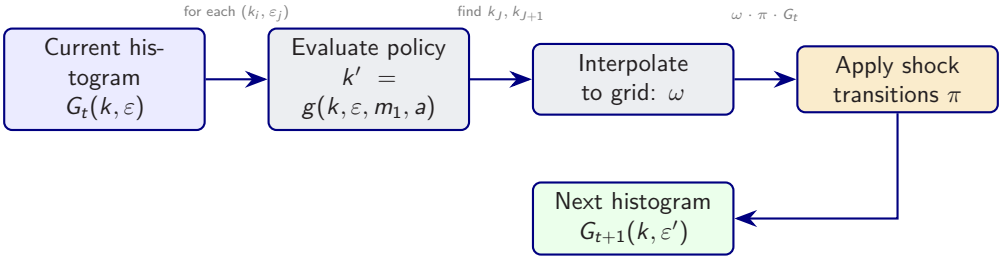
$$J(k') \;=\; \mathrm{clip}\left(\left\lfloor \frac{\log(c+k') - x_0}{\Delta x} \right\rfloor,\; 0,\; N_k - 2\right).$$

Still $\mathcal{O}(1)$ per query, no search, no branches.

Keep interpolation in k-space. The *index* is computed in x-space, but the lottery *weights* use the original k_n's, so the off-grid policy is piecewise linear in k — consistent with the on-grid consumption values and with Young's mean-preserving lottery.

Concrete implementation: `interpolate()` and `distribution_step()` in the Krusell–Smith JAX tutorial of Azinovic-Yang & Žemlička (2025), *Deep Learning in the Sequence Space*, arXiv:2509.13623.

Flow Diagram: One Forward Step



Key properties of each step:

- **Policy evaluation**: uses the solution from value function iteration (or neural net)
- **Interpolation**: linear weights ensure mean preservation
- **Shock transition**: probability-weighted mass redistribution across ε' states
- **Result**: G_{t+1} sums to 1 and has the policy-implied mean. At the fixed point this matches $\hat{H}(m_1, a)$; away from it, the outer loop closes the gap.

The Full Algorithm: Iterating to Equilibrium

Krusell–Smith Algorithm with Young's Simulation

Step 1: Initialize forecasting coefficients $A(a), B(a)$
repeat
  Step 2: Solve household problem via VFI given forecasting rule $\hat{H}$
  Step 3: Forward-simulate distribution for T periods via Young's method:
  for $t = 1, \ldots, T$ **do**
    Draw aggregate shock a_t from Markov chain
    Update histogram: $G_t \xrightarrow{\text{Young}} G_{t+1}$
    Record realized mean: $m_1^t = \sum_{i,j} G_t(k_i, \varepsilon_j) \cdot k_i$
  end for
  Step 4: Re-estimate $A(a), B(a)$ by OLS on simulated (m_1^t, m_1^{t+1}, a_t)
  Step 5: Check convergence: $|A^{\text{new}} - A^{\text{old}}| < \text{tol}$
until converged

PAUSE — 5 min Finger Exercise: Histogram Mass Redistribution

Exercise

Consider a 4-bin histogram on the grid $\{0.5, 1.0, 1.5, 2.0\}$ with initial masses $\mu = (0.25, 0.25, 0.25, 0.25)$. Suppose the policy function maps:

$$g(0.5) = 0.8, \quad g(1.0) = 1.3, \quad g(1.5) = 1.7, \quad g(2.0) = 1.9.$$

1. For $g(0.5) = 0.8$: which two grid points does 0.8 fall between? What are the interpolation weights?
2. Compute the updated mass μ' at grid point 1.0 after redistributing the mass from $g(0.5) = 0.8$ only (ignoring other bins for now).

Solution: Histogram Mass Redistribution

Answer

1. $g(0.5) = 0.8$ falls between grid points 0.5 and 1.0.
   Weight on 1.0: $\omega = \frac{0.8-0.5}{1.0-0.5} = 0.6$. Weight on 0.5: $1 - \omega = 0.4$.
2. The mass $\mu(0.5) = 0.25$ is split: grid point 1.0 receives $0.6 \times 0.25 = 0.15$, and grid point 0.5 receives $0.4 \times 0.25 = 0.10$.
   To get the full updated $\mu'(1.0)$, you would also add contributions from any other bins whose policy maps into the $[0.5, 1.5]$ interval—but this exercise only asked for the contribution from $g(0.5)$.

Total mass is preserved: $0.10 + 0.15 = 0.25 =$ original mass at bin 0.5. ✓

Convergence and Accuracy

Quality of the forecasting rule:

- $R^2 > 0.999$ for the log-linear rule $\hat{H}(m_1, a) = \exp(A(a) + B(a) \log m_1)$
- KS (1998) also tested 2- and 3-moment rules (adding variance, skewness): R^2 barely changes
- The one-moment benchmark is an empirical finding, not a modeling restriction

Accuracy tests (Den Haan, 2009):

- Compare simulated m_1 path with the forecasting rule prediction
- Maximum one-step-ahead forecast error $< 0.1\%$
- Simulation length: $T = 10{,}000$ periods (discarding first 500 for burn-in)

Convergence speed:

- Young's non-stochastic simulation removes MC noise from the simulated means
- Each iteration still includes VFI + forward simulation + OLS
- Outer-loop counts and runtime are tolerance- and implementation-dependent

Implementation: Grid Design

Practical considerations from Young (2010):

Value function grid:

- $\sim$150 points, irregularly spaced
- Finer near the borrowing constraint (high curvature)
- Coarser for large k (smooth policy)
- Used for VFI with cubic splines

Simulation grid:

- $\sim$5000 points, uniformly spaced
- Uniform spacing essential for smooth histograms
- Avoids spikes in the distribution
- Upper bound: $k_{\max} = 1200$ (large enough)

Important

Two separate grids: one for solving (irregular, fewer points), one for simulating (uniform, many points). The simulation grid must be fine and uniform to produce smooth distributions.

Boundary Cases

What happens at the grid boundaries?

- $k' \leq k_{\text{min}}$: all mass goes to the first grid point ($p = 1$)
- $k' \geq k_{\text{max}}$: all mass goes to the last grid point ($p = 0$)

Importance of grid bounds:

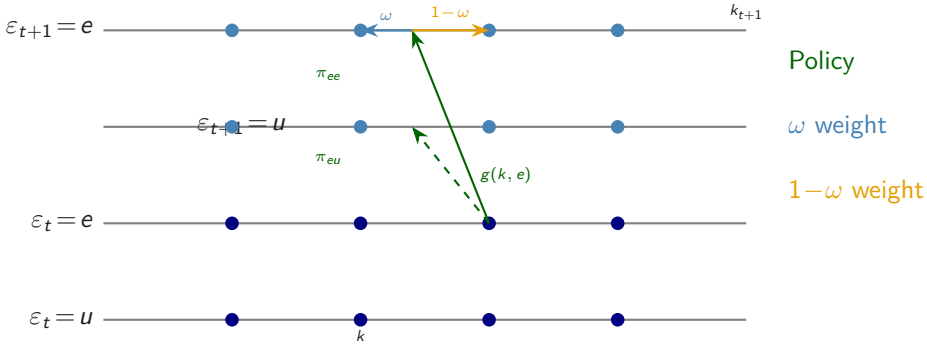
- If k_{max} is too small, mass "piles up" at the boundary $\Rightarrow$ mean is not preserved
- If k_{max} is too large, the grid wastes resolution in empty regions
- Rule of thumb: run a preliminary simulation, check that no mass reaches boundaries

From the code (Azinovic et al. 2022)

```
anext = tf.minimum(anext, amax * tf.ones_like(anext))
```

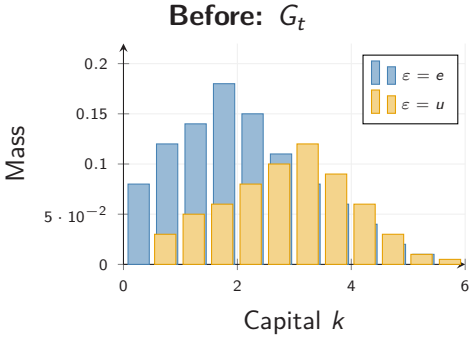
Asset holdings are clipped at $a_{\text{max}} = 20$ to prevent overflow.

Young's Simulation Procedure (Schematic)

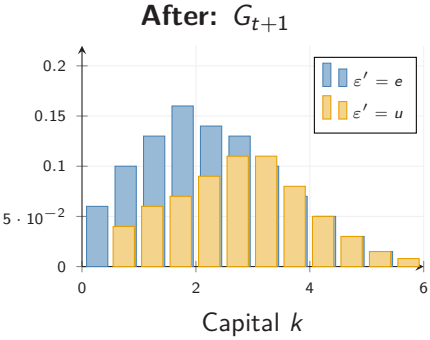


Arrows show: (1) policy maps current (k, ε) to k', (2) k' is split between grid points with weight ω, (3) mass transitions across ε' states with probabilities $\pi_{\varepsilon'|\varepsilon}$.

One Forward Step: Before and After



Mass concentrated at low k; borrowing-constrained agents at left edge



After policy + interpolation + shock transitions: mass slightly rightward

Mean preserved: $\bar{k}_t = \bar{k}_{t+1}$

Exact mean preservation!

Summary of Young's Method

Key Takeaways

1. Represents the distribution as a histogram on a fixed grid
2. Updates deterministically using policy functions + transition probabilities
3. Preserves the mean exactly; approximates higher moments
4. No sampling noise—faster and more accurate than Monte Carlo
5. Compatible with any solution method for the household problem (VFI, neural nets, . . .)

"Rather than use Monte Carlo simulation to generate an updated cross-sectional distribution . . . I use a histogram over a fixed grid to construct tomorrow's distribution."

— Young (2010, p. 38)

Part III

Benchmark: Krusell–Smith Logic

Benchmark: Krusell–Smith Logic

The Krusell–Smith (1998) economy:

- Continuum of ex ante identical, infinitely-lived agents
- Incomplete markets: no insurance against idiosyncratic risk
- Aggregate uncertainty: productivity shocks affect all agents
- Single asset: one-period bonds/capital with borrowing constraint

The challenge

Each agent must forecast future prices to make optimal savings decisions. But prices depend on the entire wealth distribution—which is an infinite-dimensional object.

$\Rightarrow$ Agents use a simple forecasting rule based on the mean of the distribution.

Model Switch: Benchmark vs. Teaching Implementation

Benchmark layer

- **Story model:** Krusell–Smith (1998)
- **Role here:** motivate approximate aggregation and the outer forecast loop
- **Young step:** deterministic distribution propagation inside KS

Teaching-code layer

- **Notebook 11:** Appendix A.5 endowment economy
- **Role here:** show Young's histogram as a DEQN state and simulator
- **Key change:** a price net replaces KS forecast-rule regression

Same operator, different shell

KS explains why distribution tracking matters. Appendix A.5 shows the same histogram update inside a DEQN.

Teaching Model: Preferences

Epstein–Zin recursive utility (separates risk aversion from IES):

$$\boxed{V(x_t) = \max_{b_{t+1} \geq 0} \left[(1-\beta)\, c_t^{1-\rho} + \beta\, \chi_t(V(x_{t+1}))^{1-\rho} \right]^{\frac{1}{1-\rho}}}$$

Certainty equivalent:

$$\chi_t(V_{t+1}) = \mathbb{E}\left[V_{t+1}^{1-\sigma} \right]^{\frac{1}{1-\sigma}}$$

Parameter	**Symbol**	**Value**
Discount factor	β	0.95
Inverse of IES	ρ	2 (IES = 0.5)
Risk aversion	σ	8

Note: $\rho \neq \sigma \Rightarrow$ agents' intertemporal substitution and risk preferences are separated.

Teaching Model: Budget and Shocks

Endowment economy (as in Azinovic et al. 2022, Appendix A.5):

- Aggregate output: $Y_t = w(a_t)$ depends on aggregate shock a_t
- Idiosyncratic labor endowment: $\eta_t \in \{0.8, 1.2\}$ with transition matrix Π_η
- Aggregate shocks: 6 states (2 uncertainty $\times$ 3 income levels)

Budget constraint:

$$\boxed{c_t + p_t \cdot b_{t+1} = b_t + \eta_t \cdot w(a_t)}$$

- b_t: bond holdings (asset)
- p_t: bond price (endogenous, clears the market)
- $\eta_t \cdot w(a_t)$: individual labor income
- Borrowing constraint: $b_{t+1} \geq 0$

Teaching Model: Individual Problem

Each agent solves:

$$V(b, \eta, x^{agg}) = \max_{b' \geq 0} \left[(1-\beta)\, c^{1-\rho} + \beta\, \chi(V(b', \eta', x'^{agg}))^{1-\rho} \right]^{\frac{1}{1-\rho}}$$

subject to:

$$c = b + \eta \cdot w(a) - p \cdot b'$$

State variables:

Individual:

- b_t: current bond holdings
- $\eta_t \in \{0.8, 1.2\}$: idiosyncratic shock

Aggregate:

- $a_t \in \{1, \ldots, 6\}$: aggregate shock
- μ_t: wealth distribution (histogram)

The bond price p_t clears the market: $\int b_{t+1}\, d\mu_t = \bar{B}$; the teaching notebook normalizes $\bar{B} = 1$ (unit net supply).

Teaching Model: Equilibrium Objects

Recursive Competitive Equilibrium

1. **Agents optimize:** given bond prices, each agent chooses b' to maximize utility
2. **Market clearing:**

$$\int b_{t+1}(b, \eta, x^{agg})\, d\mu_t(b, \eta) = \bar{B}, \qquad \bar{B} = 1 \text{ in the notebook}$$

3. **Distribution evolves consistently:** $\mu_{t+1} = T(\mu_t, a_t)$ given optimal policies
4. **Rational expectations:** agents' forecast of future prices is self-confirming

The bond price p_t:

- Depends on the aggregate state (a_t, μ_t)
- Must be found such that aggregate demand = aggregate supply = 1
- In the DEQN approach: a neural network outputs p_t directly

Traditional KS Solution Method

Step-by-step algorithm:

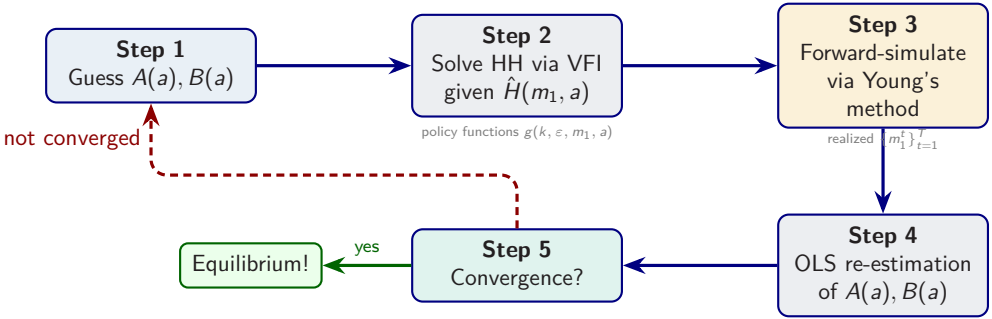
1. **Guess** forecasting rule: $p(a) \approx f(m_1, a)$
2. **Solve** household problem via VFI on (b, ε, m_1, a) grid
3. **Forward-simulate** distribution via Young's method for T periods
4. **Compute** realized m_1 and prices from simulated distributions
5. **Re-estimate** forecasting rule by regression on simulated data
6. **Check convergence**: if not converged, go to step 2

Limitations:

- VFI on fixed grid: limited resolution, curse of dimensionality
- Forecasting rule: restricted to low-order polynomial form
- Adding more moments/states is expensive

$\Rightarrow$ Can we do better with deep learning?

The Krusell–Smith Outer Loop



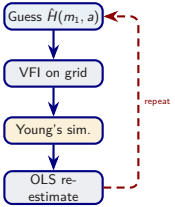
Young's simulation makes the forward-distribution step deterministic and fast, but total runtime still depends on the VFI tolerance, forecasting-rule tolerance, and implementation.

Part IV

DEQN Teaching Implementation

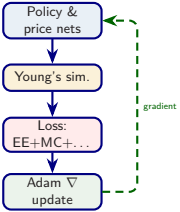
Architecture Comparison: Traditional KS vs. DEQN

Traditional KS



Outer loop; policy sees m_1 only; grid-based VFI.

DEQN + Young's



Single loop; policy sees full histogram; differentiable.

	Traditional KS	DEQN
Distribution info	mean m_1 only	full histogram
Forecasting rule	hand-specified	learned (price net)
Outer loop	implementation-dependent	none
Policy solver	VFI on grid	neural network

Teaching Model Specification (DEQN Version)

Bewley endowment economy (Azinovic et al. 2022, Appendix A.5):

Component	Value
Preferences (EZ)	$\beta = 0.95,\ \rho = 2,\ \sigma = 8$
Asset	Bonds, unit net supply
Idiosyncratic η	$\{0.8, 1.2\},\ \Pi_\eta = \begin{pmatrix} 0.7 & 0.3 \\ 0.3 & 0.7 \end{pmatrix}$
Aggregate shocks	6 states (2×3)
Asset grid	$N_b = 100,\ b \in [0, 20]$
Borrowing	$b_{t+1} \geq 0$

Budget: $c_t + p_t b_{t+1} = b_t + \eta_t w(a_t)$

Aggregate Shocks ($n_Z = 6$)

a_t	Uncert.	Income	$Y(a_t)$
0	Low	Low	0.95
1	Low	Mid	1.00
2	Low	High	1.05
3	High	Low	0.95
4	High	Mid	1.00
5	High	High	1.05

Persistence

= 0.8; product-form transition matrix.

Bridge: From Notebook 10 to Notebook 11

Concept	Notebook 10	Notebook 11
Individual asset	scalar w or k	bond holdings b
Distribution	one histogram $h(k)$	two histograms $(h^{0.8}, h^{1.2})$
Policy input	hand-set rule $g(k)$	NN policy $b'(\eta, b, x_t^{agg})$
One forward step	redistribute mass once	`get_weights_eta()` update
Aggregation	weighted mean	histogram dot product $\mathbf{h}_t \cdot \mathbf{b}'$

Reading order

Read `10_Youngs_Method_Examples` first for the operator, then `11_Continuum_of_Agents_DEQN` for the same operator inside the DEQN state vector and loss.

State Space with Histogram Encoding

Key innovation: encode μ_t as a histogram on N_b grid points.

$$\underbrace{\boxed{\eta_t}\ \boxed{b_t}}_{x_t^{id}\ (2\text{ values})}\quad \underbrace{\boxed{\begin{matrix} z_t^{idx},\ \mathrm{inc}_t^{idx} \\ \mathrm{unc}_t^{idx} \end{matrix}}\ \boxed{h_t^{\eta=0.8}\ (N_b\text{ values})}\ \boxed{h_t^{\eta=1.2}\ (N_b\text{ values})}}_{x_t^{agg}\ (3+2N_b\text{ values})}$$

$$\boxed{\text{Total input dimension} = \underbrace{2}_{\text{individual}} + \underbrace{3+2N_b}_{\text{aggregate}} = 5+2N_b}$$

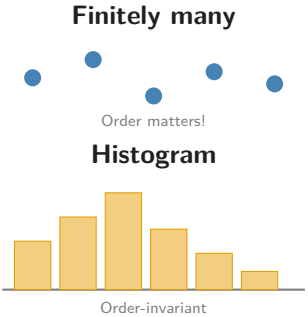
For $N_b = 100$: input dimension $= 5 + 200 =$ **205**.

The aggregate state includes the 6-state shock index, income-level index, uncertainty-regime index, and N_b histogram values for each η type.

Why Histograms? Advantages over Finitely Many Agents

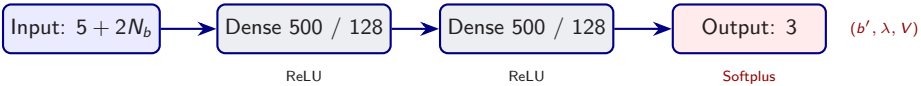
Advantages of histogram encoding

1. **No permutation issue**
   Histogram is order-invariant by construction
2. **Exact aggregation**
   $\int b' \, d\mu = \sum_j h(j) \cdot b'(j)$—no Monte Carlo noise
3. **Direct economic interpretation**
   Network "sees" the full wealth distribution
4. **Scales naturally**
   Grid resolution N_b is a tunable parameter

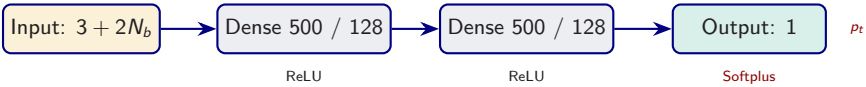


Neural Network Architecture

Policy network $\mathcal{N}_{pol}$:



Price network $\mathcal{N}_{price}$:



- **Policy net:** individual + aggregate state $\rightarrow (b', \lambda, V)$
- **Price net:** aggregate state only $\rightarrow p$
- Softplus keeps outputs strictly positive
- Paper: 500 hidden units. Teaching notebook: 128.
- Both nets are trained jointly on equilibrium residuals

Equilibrium Conditions as Loss

The DEQN solves all equilibrium conditions simultaneously (unsupervised):

1. **Euler equation (EE):** intertemporal optimality condition

$$\text{EE}: \quad \frac{\left(\frac{\beta \cdot \mathbb{E}_t[\cdots] + \lambda_t}{p_t}\right)^{-1/\rho}}{c_t} - 1 = 0$$

2. **Bellman equation (BE):** value function consistency

$$\text{BE}: \quad \frac{V^{opt}}{V^{net}} - 1 = 0$$

3. **Market clearing (MC):** aggregate bond demand = supply

$$\text{MC}: \quad \left| \sum_{\eta} \sum_{j=1}^{N_b} h_t(\eta, b_j) \cdot b'(\eta, b_j, x_t^{agg}) - \bar{B} \right| = 0$$

4. **KKT complementarity:** $\lambda_t \cdot b_{t+1} = 0$
5. **Consumption positivity:** $c_t > 0$ (penalty for violations)

Loss: Euler and Bellman Equations with Epstein–Zin

V^{opt} (Bellman residual: BE $= V^{opt}/V^{net} - 1$)

$$V_t^{opt} = \left[(1-\beta)\, c_t^{1-\rho} + \beta\, \chi_t^{1-\rho}\right]^{\frac{1}{1-\rho}}, \quad \chi_t = \left(\sum_{a',\eta'} \pi_{a'|a}\pi_{\eta'|\eta}\, V_{t+1}^{1-\sigma}\right)^{\frac{1}{1-\sigma}}$$

V_{t+1}: price network at next-period state; c_t from budget constraint.

Euler expectation:

$$\mathbb{E}_t[\cdots] = \sum_{(a',\eta')} \pi_{a'|a}\pi_{\eta'|\eta}\, c_{t+1}^{-\rho} \left(\frac{V_{t+1}}{\chi_t}\right)^{\rho-\sigma}$$

Relative Euler error (sums over all $6 \times 2 = 12$ next-period shock combinations):

$$\boxed{\mathrm{EE} = \frac{\left(\frac{\beta \cdot \mathbb{E}_t[\cdots] + \lambda_t}{p_t}\right)^{-1/\rho}}{c_t} - 1}$$

Loss: Market Clearing via Histogram

Key: Young's histogram enables exact aggregation. No Monte Carlo needed.

$$\boxed{\text{MC} = \left| \sum_{\eta \in \{0.8, 1.2\}} \sum_{j=1}^{N_b} h_t(\eta, b_j) \cdot b'(\eta, b_j, x_t^{agg}) - \bar{B} \right|}$$

- $h_t(\eta, b_j)$: mass at grid point b_j with shock η
- $b'(\eta, b_j, x_t^{agg})$: savings policy from the neural network
- Aggregate demand $= \mathbf{h}_t \cdot \mathbf{b}'$ (dot product; exact, no MC noise)
- Equilibrium: total demand $= \bar{B}$; in the notebook $\bar{B} = 1$ (unit net supply)

Implementation

The policy network is called on all $2N_b$ grid points (η, b_j) with current x_t^{agg}. Aggregate demand $= \mathbf{h}_t \cdot \mathbf{b}'$ is a dot product. Bonds are in net supply $\bar{B}$, normalized to one in the notebook.

Code: `aggdem = tf.reduce_sum(h * b_prime_grid, axis=1)` ≈ 1.

Combined Loss Function

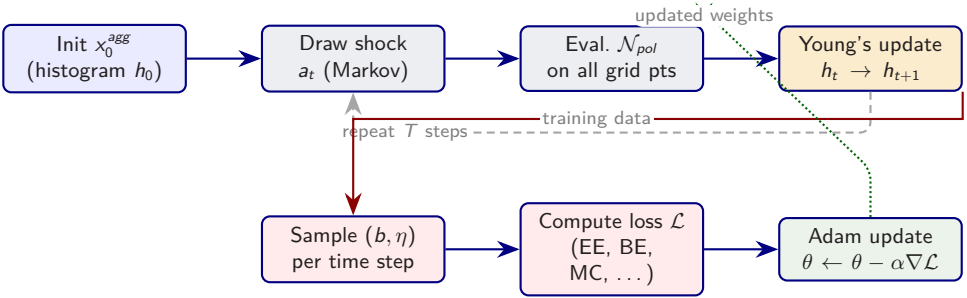
Total loss (sum of squared residuals):

$$\boxed{\mathcal{L} = \frac{1}{N}\sum_{i=1}^{N}\left[\mathrm{EE}_i^2 + \mathrm{BE}_i^2 + (n_Z \cdot \mathrm{MC}_i)^2 + \mathrm{KKT}_i^2 + \mathrm{CB}_i^2\right]}$$

Term	Condition	Interpretation
EE^2	Euler equation	Intertemporal optimality
BE^2	Bellman equation	Value function consistency
MC^2	Market clearing	Bond market equilibrium
KKT^2	$\lambda \cdot b' = 0$	Borrowing constraint complementarity
CB^2	$c > 0$	Budget feasibility penalty

- All residuals should be zero in equilibrium
- n_Z scaling on MC: ensures comparable magnitude across terms
- Minimizing $\mathcal{L}$ finds an approximate competitive equilibrium

Young's Method Inside the DEQN Training Loop



The histogram evolves during training: the network adapts to the equilibrium distribution.

Distribution Update in TensorFlow

```python
def get_weights_grid(asearch, agrid, deltaa):
    """Interpolation weights for Young's method."""
    dif = tf.abs(agrid - asearch)
    mask = tf.cast(dif < deltaa, tf.float32)
    w = (1.0 - dif / deltaa) * mask
    w = w / tf.reduce_sum(w, axis=1, keepdims=True)
    return w

def get_weights_eta(net_pol, X_agg):
    """Full histogram update using policy network."""
    h_new = tf.zeros_like(h)
    for eta_idx in range(n_eta):
        b_next = net_pol(eta, grid, X_agg)[:, 0:1]
        weights_next = get_weights_grid(b_next, agrid, deltaa)
        for eta_next in range(n_eta):
            h_new[eta_next] += pi_eta[eta_idx, eta_next] \
                               * (h[eta_idx] @ weights_next)
    h_new = normalize_per_eta(h_new, nA, target_mass=0.5)
    return h_new
```

`get_weights_grid`: interpolation weights.

`normalize_per_eta`: keeps each η block at mass 0.5.

Episode-Based Training

Training Algorithm

Input: Networks $\mathcal{N}_{pol}$, $\mathcal{N}_{price}$; Adam optimizer ($\alpha = 10^{-5}$)
Initialize histogram: equal mass at a single grid point
for episode $= 1, \ldots, N_{\text{ep}}$ **do**

1. Simulate aggregate shocks for T steps
2. Propagate histogram via Young's method using current $\mathcal{N}_{pol}$
3. For each t: sample random (b, η) pairs as idiosyncratic states
4. Form training batch: $\{(x_t^{id}, x_t^{agg}, x_{t+1}^{agg})\}$
5. Train for 1 epoch (batch size $= 128$, Adam)
6. Update starting histogram from end of simulation

end for

Notebook modes: smoke ($N_{\text{ep}} = 100$, $T = 256$), classroom (500, 256), production (65,000, 2048).

Training Design Choices

Why episode-based training?

- The policy network changes the next histogram
- The next histogram changes prices and market clearing
- So the distribution must co-evolve with the network

Practical implementation

- Default notebook mode is smoke so the full pipeline runs during class
- Carry the ending histogram into the next episode
- Use several parallel tracks for diverse starting points
- Check diagnostics on actual simulated states, not schematic plots

How to Evaluate a Real Training Run

Do not use schematic plots

Diagnostics should come from actual notebook output or from paper-reported tables and figures, not from hand-drawn convergence shapes.

1. Residuals

- Euler residuals
- Market-clearing residuals
- KKT / Bellman terms

2. Policy checks

- Monotonicity in wealth
- Borrowing limit respected
- Sensible response to income

3. Distribution checks

- Histogram remains normalized
- Mean evolves plausibly
- Prices clear given the histogram

Why Young's Method Matters Inside DEQNs

The role of Young's method in a DEQN

1. Finite-dimensional encoding of the distribution.
2. Differentiable forward simulation (backprop-compatible).
3. Exact market clearing computed from the histogram dot product.
4. The policy network is evaluated on the full $2 \times N_b$ grid, not just on sampled training states.

Takeaway

The DEQN does not replace the distributional law of motion. Young's method remains the device that turns individual decisions into an aggregate state that can be differentiated through.

Where Are We?

1. Motivation: Why Track Distributions? ✓
2. Young's (2010) Non-Stochastic Simulation ✓
3. KS benchmark + teaching-model switch ✓
4. DEQN teaching implementation ✓
5. $\Rightarrow$ Alternative deep-learning approaches to KS

Part V

Alternative Deep-Learning Approaches to Krusell–Smith

Three Deep-Learning Recipes for KS

All three recipes minimize residuals of the *same* structural equations via SGD. They differ in how the aggregate distribution is fed to the network.

	Histogram DEQN (this lecture so far)	**All-in-one DL** Maliar–Maliar–Winant	**DeepHAM** Han–Yang–E
Distribution input	fixed-grid histogram vector	explicit N-agent panel	learned generalized moments
Input dimension	$\mathcal{O}(N_b)$	$\mathcal{O}(N)$	$M \ll N_b$ (learned)
Permutation invariance	automatic	approximate (LLN)	architectural (DeepSets)
Main reference	Azinovic et al. (2022)	Maliar, Maliar & Winant (2021)	Han, Yang & E (2023)

The accompanying notebook `12_KrusellSmith_DeepLearning.ipynb` implements the middle column at classroom scale.

All-in-One Deep Learning (Maliar, Maliar & Winant 2021)

Idea. Cast every equilibrium condition as a *residual that should vanish*, and train deep networks by minimizing the squared residual on simulated paths.

Loss components (for Krusell–Smith)

- **Euler residual:** $\mathrm{EE} = u'(c_t) - \beta\,\mathbb{E}[u'(c_{t+1})R_{t+1}]$, Monte Carlo over next-period shocks.
- **Bellman residual** on an auxiliary value network.
- **Market clearing:** $\sum_{i=1}^{N} k_{t+1}^{i} = N\bar{K}_{t+1}$.
- **Borrowing constraint:** softplus on savings head or Fischer–Burmeister residual.

Reference: Maliar, Maliar & Winant (2021), *J. Monet. Econ.* 122, 76–101.

All-in-One DL: Scale and Replication

Scale reported by the authors.

- Up to $N = 10{,}000$ explicit agents.
- State dimension 2,001+ (one per agent plus aggregate).
- Approximation errors below 1% on consumption-saving.

Replication code.
`github.com/marcmaliar/deep-learning-euler-method-krusell-smith`

In this course. Notebook `12_KrusellSmith_DeepLearning.ipynb` is a classroom-scale caricature of this approach: single policy network, panel of $N \sim 100$ agents, CPU-friendly, to convergence.

DeepHAM — Learned Moments (Han, Yang & E)

Idea. Instead of tracking the histogram or committing to the first moment *a priori*, learn which summary statistics agents actually need.

Generalized moments via DeepSets

$$\boldsymbol{m}_t = \left(\sum_i g_\theta^1(s_t^i),\ \ldots,\ \sum_i g_\theta^M(s_t^i) \right), \qquad M \ll N_b$$

Each $g_\theta^m : \mathbb{R}^d \to \mathbb{R}$ is a neural encoder trained *jointly* with the value and policy networks. Sum-pooling $\Rightarrow$ permutation invariance.

Reference: Han, Yang & E (2024), *Quantitative Economics* (forthcoming); preprint arXiv:2112.14377.

DeepHAM: Reported Accuracy and Extensions

Results on baseline KS:

- First moment only: Bellman error reduced by 37.5% vs. classical KS.
- With one *learned* moment: Bellman error reduced by 54.2%.

Extensions where DeepHAM shines.

- Multiple endogenous states (capital + bonds).
- Large aggregate shocks (risky steady state).
- Zero lower bound; occasionally binding constraints that bite on a non-trivial mass.

These are settings where both classical KS and the local perturbation method of Reiter (2009) break down.

Head-to-Head: When to Use Which?

	Pros	When it shines
Histogram DEQN (Azinovic et al., 2022)	Interpretable state; exact market clearing; DEQN-template reuse	Teaching; N_b moderate; smooth policies
All-in-one DL (Maliar, Maliar & Winant, 2021)	No grid design; large-N panels; single optimizer for all residuals	Large-N research problems; OLG-many-cohort extensions
DeepHAM (Han, Yang & E, 2023)	Learned moments; risky steady state; ZLB; multiple endogenous states	Rich HA macro-finance with nonlinear aggregate effects

Classical baseline

The canonical traditional KS implementation (`econ-ark/KrusellSmith` REMARK) achieves $R^2 > 0.9999$ in ~ 20 outer iterations on the baseline model. All deep-learning methods aim to match or beat this, and to extend beyond the benchmark where KS can't.

Replication Codes + Where to Get Them

Approach	**Repository (on GitHub)**
Traditional KS (baseline)	`econ-ark/KrusellSmith` *REMARK replication; forecasting-rule algorithm.*
All-in-one DL (MMW 2021)	`marcmaliar/deep-learning-euler-method-krusell-smith` *TensorFlow implementation accompanying the JME paper.*
Histogram DEQN	`sischei/DeepEquilibriumNets` *Azinovic, Gaegauf & Scheidegger (2022) code.*
Classroom MMW-style (this course)	`lecture_10_B08/code/12_KrusellSmith_DeepLearning.ipynb` *Self-contained, CPU-friendly, training.*
DeepHAM	Authors' page; contact the authors for code.

Summary

Key Messages

1. **HA models** require tracking the wealth distribution, an infinite-dimensional state variable.
2. **Young (2010)** gives a deterministic histogram update: no Monte Carlo noise, exact mean preservation.
3. **Krusell–Smith** is the benchmark layer: it explains approximate aggregation and the forecasting-rule loop.
4. **Notebook 11** is the teaching-code layer: Appendix A.5 reuses Young's method inside a DEQN with policy and price networks.
5. **Young + DEQN** keeps exact histogram aggregation while removing the separate KS regression step.

Core References

- Azinovic, M., Gaegauf, L., & Scheidegger, S. (2022). Deep equilibrium nets. *Int. Econ. Rev.* 63(4), 1471–1525.
- Den Haan, W. (2010). Assessing the accuracy of the aggregate law of motion in models with heterogeneous agents. *J. Econ. Dyn. Control* 34(1), 79–99.
- Han, J., Yang, Y., & E, W. (2024). DeepHAM: A global solution method for HA models with aggregate shocks. *Quant. Econ.* (forthcoming); arXiv:2112.14377.
- Krusell, P., & Smith, A.A. (1998). Income and wealth heterogeneity in the macroeconomy. *J. Polit. Econ.* 106(5), 867–896.
- Maliar, L., Maliar, S., & Winant, P. (2021). Deep learning for solving dynamic economic models. *J. Monet. Econ.* 122, 76–101.
- Maliar, L., Maliar, S., & Valli, F. (2010). *J. Econ. Dyn. Control* 34(1), 42–49.
- Reiter, M. (2009). Solving heterogeneous-agent models by projection and perturbation. *J. Econ. Dyn. Control* 33(3), 649–665.
- Young, E.R. (2010). Solving the incomplete markets model with aggregate uncertainty . . . *J. Econ. Dyn. Control* 34(1), 36–41.

Bold-face entries are the two alternative DL methods covered in Part V and in the reading guide for this lecture.

Core Notebook Map (Discrete-Time Block)

Core (Parts II–IV)

`10_Youngs_Method_Examples`

`11_Continuum_of_Agents_DEQN`

Alternative DL (Part V)

`12_KrusellSmith_DeepLearning`
All-in-one DL; classroom scale

All notebooks are in the course repository.

Sequence-space extension

`05_SequenceSpace_BrockMirman`

`06_SequenceSpace_KrusellSmith`

External reference code

`econ-ark/KrusellSmith`

`marcmaliar/deep-learning-euler-method-krusell-smith`

Questions?

Simon Scheidegger

HEC, University of Lausanne

https://sischei.github.io

Materials:

https://github.com/sischei/Deep_Learning_for_Solving_And_Estimating_Dynamic_Economic_Models

Next up:

Agentic Programming

Using AI agents for research

PINNs & continuous-time HA

Surrogates & GPs

Climate & Deep UQ